第八章 用户进程、线程与 libc

本章内容

- 用户程序的形态与系统调用网关
- 从 init 到 shell
- 用户程序一览
- 用户线程：clone、texit、tjoin
- libc 与 POSIX 源码兼容层
- 在 xv6 里编译 C 程序
- 一个大型用户程序：sshd

8.1 用户程序的形态

8.1.1 为什么需要系统调用

进程隔离的第一步，是把所有用户程序放到特权最低的 EL0：在这一级，程序不能修改页表基址、不能屏蔽中断、不能访问没有映射给它的内存，也就无法改写内核为隔离所做的任何设置。否则，其他一切隔离手段都没有意义。

可是用户程序总要打印字符、读文件、发网络包，而这些都需要直接操作设备。我们不能让每个程序直接去写 UART 寄存器，而是由内核提供一组受控的入口——系统调用。它不能是普通的函数调用，因为每次调用都要把执行级别从 EL0 提升到 EL1，而提升级别只有一条路：产生异常。所以每个系统调用本质上都是一次同步异常：用户程序准备好参数，执行 svc 指令；硬件把它带到 EL1 的异常向量；内核检查参数、完成请求，然后执行普通的异常返回，从 svc 的下一条指令继续。进程隔离的第三个方面——每个进程有自己独立的内存视图——由页表完成（第 4 章）。

8.1.2 链接与装载

xv6 原版把用户程序和用户库静态链接成一个 ELF64 文件，链接在虚拟地址 0，入口是 main。本项目在 make DYNAMIC_USER=0 时仍退回这一形态（默认形态改为动态链接，见第 8.5.2 节）：

```
_%: %.o $(ULIB)
    $(LD) $(LDFLAGS) -N -e main -Ttext 0 -o $@ $^
```

-N 让代码段和数据段放在同一个可读写可执行的段里，exec() 只需处理一个 PT_LOAD，也没有 C 运行时启动代码：exec() 把 argc、argv 放进 x0、x1，把 PC 设为入口地址，main 返回后由用户库调用 exit()——实际上 xv6 的程序习惯在 main 末尾显式调用 exit(0)。这一形态简单，但代价是每个程序都各自携带一份 libc 的代码；把用户库改成共享的 libc.so、让程序默认以位置无关（PIE）ET_DYN 形式动态链接它，就是第 8.5.2 节的内容。

8.1.3 系统调用网关

user/usys.pl 只生成一个私有陷入网关 __syscall，而不是每个系统调用一条桩。它把系统调用号从第一个参数 x0 挪进 x7，其余参数整体下移一格，然后陷入 EL1：

```
.global __syscall
__syscall:
 mov x7, x0      // 第 0 个实参是系统调用号
 mov x0, x1      // 真正的参数 x1..x6 下移成 x0..x5
```